

Design

- Before Fixed Bugs

```
 8 module FIFO(data_in, wr_en, rd_en, clk, rst_n, full, empty, almostfull, almostempty, wr_ack, overflow, underflow, data_out);
 9 parameter FIFO_WIDTH = 16;
10 parameter FIFO_DEPTH = 8;
11 input [FIFO_WIDTH-1:0] data_in;
12 input clk, rst_n, wr_en, rd_en;
13 output reg [FIFO_WIDTH-1:0] data_out;
14 output reg wr_ack, overflow;
15 output full, empty, almostfull, almostempty, underflow;
16
17 localparam max_fifo_addr = $clog2(FIFO_DEPTH);
18
19 reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
20
21 reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
22 reg [max_fifo_addr:0] count;
23
24 always @(posedge clk or negedge rst_n) begin
25     if (!rst_n) begin
26         wr_ptr <= 0;
27     end
28     else if (wr_en && count < FIFO_DEPTH) begin
29         mem[wr_ptr] <= data_in;
30         wr_ack <= 1;
31         wr_ptr <= wr_ptr + 1;
32     end
33     else begin
34         wr_ack <= 0;
35         if (full & wr_en)
36             overflow <= 1;
37         else
38             overflow <= 0;
39     end
40 end
41
42 always @(posedge clk or negedge rst_n) begin
43     if (!rst_n) begin
44         rd_ptr <= 0;
45     end
46     else if (rd_en && count != 0) begin
47         data_out <= mem[rd_ptr];
48         rd_ptr <= rd_ptr + 1;
49     end
50 end
51
52 always @(posedge clk or negedge rst_n) begin
53     if (!rst_n) begin
54         count <= 0;
55     end
56     else begin
57         if ( ({wr_en, rd_en} == 2'b10) && !full)
58             count <= count + 1;
59         else if ( ({wr_en, rd_en} == 2'b01) && !empty)
60             count <= count - 1;
61     end
62 end
63
64 assign full = (count == FIFO_DEPTH)? 1 : 0;
65 assign empty = (count == 0)? 1 : 0;
66 assign underflow = (empty && rd_en)? 1 : 0;
67 assign almostfull = (count == FIFO_DEPTH-2)? 1 : 0;
68 assign almostempty = (count == 1)? 1 : 0;
69
70 endmodule
```

- Design after Fixed Bugs

```
1 ///////////////////////////////////////////////////////////////////
2 // Author: Kareem Waseem
3 // Course: Digital Verification using SV & UVM
4 //
5 // Description: FIFO Design
6 //
7 ///////////////////////////////////////////////////////////////////
8 module FIFO(fifo_if.DUT f_if);
9
10
11 localparam max_fifo_addr = $clog2(f_if.FIFO_DEPTH);
12
13 reg [f_if.FIFO_WIDTH-1:0] mem [f_if.FIFO_DEPTH-1:0];
14
15 reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
16 reg [max_fifo_addr:0] count;
17
18 //-----
19 // Write Operation
20 //-----
21 always @(posedge f_if.clk or negedge f_if.rst_n) begin
22     if (!f_if.rst_n) begin
23         wr_ptr <= 0;
24         f_if.wr_ack <= 0; //Fixed Bug: Added Reset value to wr_ack
25         f_if.overflow <= 0; //Fixed Bug: Added Reset value to overflow
26     end
27     else if (f_if.wr_en && count < f_if.FIFO_DEPTH) begin
28         mem[wr_ptr] <= f_if.data_in;
29         f_if.wr_ack <= 1;
30         wr_ptr <= wr_ptr + 1;
31     end
32     else begin
33         f_if.wr_ack <= 0;
34         if (f_if.full & f_if.wr_en)
35             f_if.overflow <= 1;
36         else
37             f_if.overflow <= 0;
38     end
39 end
40 //-----
41 // Read Operation
42 //-----
43 always @(posedge f_if.clk or negedge f_if.rst_n) begin
44     if (!f_if.rst_n) begin
45         f_if.data_out <= 0; //Fixed Bug: was f_if.data_out <= 0;
46         rd_ptr <= 0;
47         f_if.underflow <= 0; //Fixed Bug: Added Reset value to underflow
48     end
49     else if (f_if.rd_en && count != 0) begin
50         f_if.data_out <= mem[rd_ptr];
51         rd_ptr <= rd_ptr + 1;
52     end
53     else begin
54         if (f_if.empty & f_if.rd_en)
55             f_if.underflow <= 1; //Fixed Bug: Underflow should be sequential
56         else
57             f_if.underflow <= 0;
58     end
59 end
60
61 //-----
62 // Status Flags
63 //-----
64 always @(posedge f_if.clk or negedge f_if.rst_n) begin
65     if (!f_if.rst_n) begin
66         count <= 0;
67     end
68     else begin
69         if ((f_if.wr_en, f_if.rd_en) == 2'b10) && !f_if.full
70             count <= count + 1;
71         else if ((f_if.wr_en, f_if.rd_en) == 2'b01) && !f_if.empty
72             count <= count - 1;
73         else if ((f_if.wr_en, f_if.rd_en) == 2'b11) && f_if.full //Fixed Bug: Added cases when wr_en and rd_en are high simultaneously:
74             count <= count - 1;
75         else if ((f_if.wr_en, f_if.rd_en) == 2'b11) && f_if.empty //Fixed Bug: Added cases when wr_en and rd_en are high simultaneously:
76             count <= count + 1;
77     end
78 end
79
80 assign f_if.full = (count == f_if.FIFO_DEPTH)? 1 : 0;
81 assign f_if.empty = (count == 0)? 1 : 0;
82 assign f_if.almostfull = (count == f_if.FIFO_DEPTH-1)? 1 : 0; //Fixed Bug: was f_if.FIFO_DEPTH - 1
83 assign f_if.almostempty = (count == 1)? 1 : 0;
```

Fixed Bugs:

Design after fixing and Assertions:

- The 'overflow' and 'wr_ack' should be reset.
- The underflow signal is supposed to be sequential output not combinational and changing its type to output only not reg.
- The almost full signal is supposed to be high when the count is below the depth of the FIFO by one not two.
- The count should check also for cases when the write and read enables are high while it's empty or full, as one operation should take place then the counter will increment or decrement.

Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functional Checking
FIFO_1	When reset is asserted the output of the FIFO should be low	Directed at the beginning of the simulations	N/A	A checker to make sure output is correct through reference model and assertions
FIFO_2	When rd_en=1 and empty=1 the underflow flag should be asserted	Directed after FIFO_1	Covers cross_wr_rd_full,cross_wr_rd_almostfull,cross_wr_rd_empty,cross_wr_rd_almostempty,cross_wr_rd_overflow,cross_wr_rd_underflow,cross_wr_rd_wr_ack	A checker to make sure output is correct through reference model and assertions
FIFO_3	Random tests to check for the full functionality of the FIFO	Directed after FIFO_2 , random tests with constraints on rd_en and wr_en	Covers cross_wr_rd_full,cross_wr_rd_almostfull,cross_wr_rd_empty,cross_wr_rd_almostempty,cross_wr_rd_overflow,cross_wr_rd_underflow,cross_wr_rd_wr_ack	A checker to make sure output is correct through reference model and assertions

Assertions:

```
86 //=====
87 //=====Assertions=====
88 //=====
89
90 // module FIFO_SVA(fifo_if.monitor f_if);
91 //a. Reset Behavior
92 always_comb begin
93     if(~f_if.rst_n)begin
94         assert final (f_if.full == 0);
95         cover final (f_if.full == 0);
96
97         assert final (f_if.empty == 1);
98         cover final (f_if.empty == 1);
99
100         assert final (f_if.almostfull == 0);
101         cover final (f_if.almostfull == 0);
102
103         assert final (f_if.almostempty == 0);
104         cover final (f_if.almostempty == 0);
105
106         assert final (wr_ptr == 0);
107         cover final (wr_ptr == 0);
108
109         assert final (rd_ptr == 0);
110         cover final (rd_ptr == 0);
111
112         assert final (count == 0);
113         cover final (count == 0);
114
115     end
116
117 end
118
119 //b. Write Acknowledge (wr_ack)
120 property p_wr_ack;
121     @(posedge f_if.clk) disable iff(~f_if.rst_n) (f_if.wr_en && !f_if.full) |>= (f_if.wr_ack);
122 endproperty
123
124 //c. Overflow Detection
125 property p_overflow;
126     @(posedge f_if.clk) disable iff(~f_if.rst_n) (f_if.wr_en && f_if.full) |>= (f_if.overflow);
127 endproperty
128
129 //d. Underflow Detection
130 property p_underflow;
131     @(posedge f_if.clk) disable iff(~f_if.rst_n) (f_if.rd_en && f_if.empty) |>= (f_if.underflow);
132 endproperty
133
134 //e. Empty Flag Assertion
135 property p_empty;
136     @(posedge f_if.clk) disable iff(~f_if.rst_n) (count == 0) |>= (f_if.empty);
137 endproperty
138
139 //f. Full Flag Assertion
140 property p_full;
141     @(posedge f_if.clk) disable iff(~f_if.rst_n) (count == f_if.FIFO_DEPTH) |>= (f_if.full);
142 endproperty
143
144 //g. Almost Full Condition
145 property p_almostfull;
146     @(posedge f_if.clk) disable iff(~f_if.rst_n) (count == f_if.FIFO_DEPTH - 1) |>= (f_if.almostfull);
147 endproperty
148
149 //h. Almost Empty Condition
150 property p_almostempty;
151     @(posedge f_if.clk) disable iff(~f_if.rst_n) (count == 1) |>= (f_if.almostempty);
152 endproperty
153
154 //i. Pointer Wraparound
155 property p_wr_ptr_wraparound;
156     @(posedge f_if.clk) disable iff(~f_if.rst_n) (f_if.wr_en && !f_if.full && wr_ptr == f_if.FIFO_DEPTH-1) |>= (wr_ptr == 0);
157 endproperty
158
159 property p_rd_ptr_wraparound;
160     @(posedge f_if.clk) disable iff(~f_if.rst_n) (f_if.rd_en && !f_if.empty && rd_ptr == f_if.FIFO_DEPTH-1) |>= (rd_ptr == 0);
161 endproperty
162
163 property p_counter_wraparound;
164     @(posedge f_if.clk) disable iff(~f_if.rst_n)
165     (count == f_if.FIFO_DEPTH && f_if.wr_en && !f_if.full && wr_ptr == f_if.FIFO_DEPTH-1) |>= (count == 0);
166 endproperty
167
168 //j. Pointer threshold
169 property p_wr_ptr_threshold;
170     @(posedge f_if.clk) (wr_ptr < f_if.FIFO_DEPTH);
171 endproperty
172
173 property p_rd_ptr_threshold;
174     @(posedge f_if.clk) (rd_ptr < f_if.FIFO_DEPTH);
175 endproperty
176
177 property p_count_threshold;
178     @(posedge f_if.clk) (count <= f_if.FIFO_DEPTH);
179 endproperty
180
```

```

179     endproperty
180
181     //=====
182     // FIFO Assertion Properties & Coverage
183     //=====
184
185     // b. Write Acknowledge (wr_ack)
186     p_wr_ack_assertion      : assert property (p_wr_ack);
187     p_wr_ack_coverage      : cover  property (p_wr_ack);
188
189     // c. Overflow Detection
190     p_overflow_assertion    : assert property (p_overflow);
191     p_overflow_coverage     : cover  property (p_overflow);
192
193     // d. Underflow Detection
194     p_underflow_assertion   : assert property (p_underflow);
195     p_underflow_coverage    : cover  property (p_underflow);
196
197     // e. Empty Flag Assertion
198     p_empty_assertion       : assert property (p_empty);
199     p_empty_coverage        : cover  property (p_empty);
200
201     // f. Full Flag Assertion
202     p_full_assertion        : assert property (p_full);
203     p_full_coverage        : cover  property (p_full);
204
205     // g. Almost Full Condition
206     p_almostfull_assertion  : assert property (p_almostfull);
207     p_almostfull_coverage  : cover  property (p_almostfull);
208
209     // h. Almost Empty Condition
210     p_almostempty_assertion : assert property (p_almostempty);
211     p_almostempty_coverage  : cover  property (p_almostempty);
212
213     // i. Pointer Wraparound (Write)
214     p_wr_ptr_wraparound_assertion : assert property (p_wr_ptr_wraparound);
215     p_wr_ptr_wraparound_coverage  : cover  property (p_wr_ptr_wraparound);
216
217     // i. Pointer Wraparound (Read)
218     p_rd_ptr_wraparound_assertion : assert property (p_rd_ptr_wraparound);
219     p_rd_ptr_wraparound_coverage  : cover  property (p_rd_ptr_wraparound);
220
221     // j. Pointer Threshold (Write)
222     p_wr_ptr_thresh_assertion : assert property (p_wr_ptr_thresh);
223     p_wr_ptr_thresh_coverage  : cover  property (p_wr_ptr_thresh);
224
225     // j. Pointer Threshold (Read)
226     p_rd_ptr_thresh_assertion : assert property (p_rd_ptr_thresh);
227     p_rd_ptr_thresh_coverage  : cover  property (p_rd_ptr_thresh);
228
229     // j. Count Threshold
230     p_count_thresh_assertion : assert property (p_count_thresh);
231     p_count_thresh_coverage  : cover  property (p_count_thresh);
232 `endif
233 endmodule

```

Shared Package:



```

1  package shared_pkg;
2
3      bit test_finished = 0;
4      int error_count = 0, correct_count = 0;
5
6  endpackage

```

Interface

```
1 import shared_pkg::*;
2
3 interface fifo_if(input clk);
4
5     parameter FIFO_WIDTH = 16;
6     parameter FIFO_DEPTH = 8;
7     event driving_finished, monitor_finished;
8
9     logic [FIFO_WIDTH-1:0] data_in;
10    logic rst_n, wr_en, rd_en;
11    logic [FIFO_WIDTH-1:0] data_out;
12    logic wr_ack, overflow;
13    logic full, empty, almostfull, almostempty, underflow;
14
15
16    //Modports
17    modport DUT (input clk, rst_n, wr_en, rd_en, data_in,
18                output data_out, full, empty, almostfull, almostempty, wr_ack, overflow, underflow);
19
20    modport test (input clk, data_out, full, empty, almostfull, almostempty, wr_ack, overflow, underflow, driving_finished, monitor_finished,
21                 output rst_n, wr_en, rd_en, data_in);
22
23    modport monitor (input clk, rst_n, wr_en, rd_en, data_in, driving_finished, monitor_finished,
24                    data_out, full, empty, almostfull, almostempty, wr_ack, overflow, underflow);
25
26 endinterface
```

Transaction package

```
1 package FIFO_transaction_pkg;
2
3     import shared_pkg::*;
4
5     class FIFO_transaction #(parameter FIFO_WIDTH = 16, parameter FIFO_DEPTH = 8);
6
7         //Inputs
8         rand logic [FIFO_WIDTH-1:0] data_in;
9         rand logic rst_n, wr_en, rd_en;
10        //Outputs
11        logic [FIFO_WIDTH-1:0] data_out;
12        logic wr_ack, overflow;
13        logic full, empty, almostfull, almostempty, underflow;
14
15        int RD_EN_ON_DIST, WR_EN_ON_DIST;
16
17        function new(RD_EN_ON_DIST = 30, WR_EN_ON_DIST = 70);
18            this.RD_EN_ON_DIST = 30;
19            this.WR_EN_ON_DIST = 70;
20        endfunction
21
22        constraint c_rst_n{
23            rst_n dist{1:/90, 0:/10};
24        }
25
26        constraint c_rd_en{
27            rd_en dist{1:/RD_EN_ON_DIST, 0:/((100 - RD_EN_ON_DIST))};
28        }
29
30        constraint c_wr_en{
31            wr_en dist {1:/WR_EN_ON_DIST, 0:/((100 - WR_EN_ON_DIST))};
32        }
33    endclass
34
35 endpackage
```

Scoreboard Package

```
1 package FIFO_scoreboard_pkg;
2
3 import shared_pkg::*;
4 import FIFO_transaction_pkg::*;
5
6 class FIFO_scoreboard #(parameter FIFO_WIDTH = 16, parameter FIFO_DEPTH = 8);
7
8     bit [FIFO_WIDTH-1:0] data_out_ref;
9
10    reg [FIFO_WIDTH-1:0] fifo_q [$]; // dynamic queue to mimic FIFO memory
11
12    function new();
13        fifo_q.delete();
14    endfunction
15
16
17    function void check_data(FIFO_transaction transaction_chk);
18
19        //call reference_model(transaction_chk)
20        reference_model(transaction_chk);
21
22        //compare DUT outputs with reference model outputs
23        if( transaction_chk.data_out != data_out_ref) begin
24            $display("time: %0t Data Mismatch! Expected: %0h, Got: %0h", $time,data_out_ref, transaction_chk.data_out);
25            error_count++;
26        end
27        else begin
28            correct_count++;
29        end
30
31    endfunction
32
33    function void reference_model(input FIFO_transaction transaction_chk);
34        //assign correct value to ref_signals
35        // -----
36        //Reset Sequence
37        // -----
38        if(!transaction_chk.rst_n)begin
39            fifo_q.delete();
40            data_out_ref = 0;
41        end
42        // Handle Read Operation
43        else if({transaction_chk.wr_en, transaction_chk.rd_en} == 2'b01 && fifo_q.size() > 0)begin
44            data_out_ref = fifo_q.pop_front();
45        end
46        // Handle Write Operation
47        else if({transaction_chk.wr_en, transaction_chk.rd_en} == 2'b10 && fifo_q.size() < FIFO_DEPTH)begin
48            fifo_q.push_back(transaction_chk.data_in);
49        end
50        // Handle Write/Read Operation
51        else if ({transaction_chk.wr_en, transaction_chk.rd_en} == 2'b11) begin
52            if(fifo_q.size() == FIFO_DEPTH)
53                data_out_ref = fifo_q.pop_front();
54            else(fifo_q.size() == 0)
55                fifo_q.push_back(transaction_chk.data_in);
56            else begin
57                data_out_ref = fifo_q.pop_front();
58                fifo_q.push_back(transaction_chk.data_in);
59            end
60        end
61
62    endfunction
63
64 endclass
65 endpackage
```

Coverage Package

```
1  package FIFO_coverage_pkg;
2
3      import shared_pkg::*;
4      import FIFO_transaction_pkg::*;
5
6      class FIFO_coverage;
7
8          FIFO_transaction F_cvg_txn;
9
10         covergroup CovCode;
11             cp_wr_en    : coverpoint F_cvg_txn.wr_en;
12             cp_rd_en    : coverpoint F_cvg_txn.rd_en;
13
14             // output control signals
15             cp_full      : coverpoint F_cvg_txn.full;
16             cp_almostfull : coverpoint F_cvg_txn.almostfull;
17             cp_empty      : coverpoint F_cvg_txn.empty;
18             cp_almostempty : coverpoint F_cvg_txn.almostempty;
19             cp_overflow    : coverpoint F_cvg_txn.overflow;
20             cp_underflow   : coverpoint F_cvg_txn.underflow;
21             cp_wr_ack      : coverpoint F_cvg_txn.wr_ack;
22
23             // 7 required crosses (wr_en x rd_en x each output control)
24             // 1. Full flag cross
25             cross_wr_rd_full : cross cp_wr_en, cp_rd_en, cp_full {
26                 ignore_bins write_when_full_and_rd =
27                     binsof(cp_wr_en) intersect {1} &&
28                     binsof(cp_rd_en) intersect {1} &&
29                     binsof(cp_full) intersect {1}; // overflow case handled separately
30
31                 ignore_bins read_when_full_and_wr_low =
32                     binsof(cp_wr_en) intersect {0} &&
33                     binsof(cp_rd_en) intersect {1} &&
34                     binsof(cp_full) intersect {1};
35             }
36
37             // 2. Almost full flag cross
38             cross_wr_rd_almostfull : cross cp_wr_en, cp_rd_en, cp_almostfull;
39
40             // 3. Empty flag cross
41             cross_wr_rd_empty : cross cp_wr_en, cp_rd_en, cp_empty {
42                 ignore_bins read_when_empty_and_wr =
43                     binsof(cp_wr_en) intersect {1} &&
44                     binsof(cp_rd_en) intersect {1} &&
45                     binsof(cp_empty) intersect {1}; // underflow case handled separately
46
47                 ignore_bins write_when_empty_and_rd_low =
48                     binsof(cp_wr_en) intersect {1} &&
49                     binsof(cp_rd_en) intersect {0} &&
50                     binsof(cp_empty) intersect {1};
51             }
52
```



```

52
53     // 4. Almost empty flag cross
54     cross_wr_rd_almostempty : cross cp_wr_en, cp_rd_en, cp_almostempty;
55
56     // 5. Overflow signal cross
57     cross_wr_rd_overflow : cross cp_wr_en, cp_rd_en, cp_overflow {
58         ignore_bins impossible_no_write =
59             binsof(cp_wr_en) intersect {0} &&
60             binsof(cp_overflow) intersect {1};
61         ignore_bins read_only_overflow =
62             binsof(cp_rd_en) intersect {1} &&
63             binsof(cp_overflow) intersect {1};
64     }
65
66     // 6. Underflow signal cross
67     cross_wr_rd_underflow : cross cp_wr_en, cp_rd_en, cp_underflow {
68         ignore_bins impossible_no_read =
69             binsof(cp_rd_en) intersect {0} &&
70             binsof(cp_underflow) intersect {1};
71         ignore_bins write_only_underflow =
72             binsof(cp_wr_en) intersect {1} &&
73             binsof(cp_underflow) intersect {1};
74     }
75
76     // 7. Write Acknowledge cross
77     cross_wr_rd_wr_ack : cross cp_wr_en, cp_rd_en, cp_wr_ack {
78         ignore_bins wr_ack_without_write =
79             binsof(cp_wr_en) intersect {0} &&
80             binsof(cp_wr_ack) intersect {1};
81     }
82
83
84     endgroup
85
86     function new();
87         F_cvg_txn = new;
88         CovCode = new;
89     endfunction
90
91     function void sample_data(FIFO_transaction F_txn);
92         this.F_cvg_txn.data_in      = F_txn.data_in;
93         this.F_cvg_txn.rst_n        = F_txn.rst_n;
94         this.F_cvg_txn.wr_en        = F_txn.wr_en;
95         this.F_cvg_txn.rd_en        = F_txn.rd_en;
96         this.F_cvg_txn.data_out     = F_txn.data_out;
97         this.F_cvg_txn.wr_ack       = F_txn.wr_ack;
98         this.F_cvg_txn.overflow     = F_txn.overflow;
99         this.F_cvg_txn.underflow    = F_txn.underflow;
100        this.F_cvg_txn.full         = F_txn.full;
101        this.F_cvg_txn.empty        = F_txn.empty;
102        this.F_cvg_txn.almostfull   = F_txn.almostfull;
103        this.F_cvg_txn.almostempty  = F_txn.almostempty;
104        CovCode.sample();
105    endfunction
106 endclass
107
108 endpackage

```

Monitor

```
1 import shared_pkg::*;
2 import FIFO_transaction_pkg::*;
3 import FIFO_coverage_pkg::*;
4 import FIFO_scoreboard_pkg::*;
5
6 module fifo_monitor(fifo_if.monitor f_if);
7
8     FIFO_transaction transaction_obj;
9     FIFO_scoreboard scoreboard_obj;
10    FIFO_coverage coverage_obj;
11    initial begin
12
13        transaction_obj = new;
14        scoreboard_obj = new;
15        coverage_obj = new;
16        @(negedge f_if.clk);
17
18        forever begin
19            @(negedge f_if.clk);
20            wait(f_if.driving_finished.triggered);
21            //sample data form interface and put inside transactions_obj
22            transaction_obj.data_in = f_if.data_in;
23            transaction_obj.rst_n = f_if.rst_n;
24            transaction_obj.wr_en = f_if.wr_en;
25            transaction_obj.rd_en = f_if.rd_en;
26            transaction_obj.data_out = f_if.data_out;
27            transaction_obj.wr_ack = f_if.wr_ack;
28            transaction_obj.overflow = f_if.overflow;
29            transaction_obj.underflow = f_if.underflow;
30            transaction_obj.full = f_if.full;
31            transaction_obj.empty = f_if.empty;
32            transaction_obj.almostfull = f_if.almostfull;
33            transaction_obj.almostempty = f_if.almostempty;
34            ->f_if.driving_finished;
35            fork
36                //Process-1 (Sample Data) Sampling for coverage
37                begin
38                    coverage_obj.sample_data(transaction_obj);
39                end
40                //Process-2 (Check data with scoreboard)
41                begin
42                    scoreboard_obj.check_data(transaction_obj);
43                end
44            join
45
46            if(test_finished)begin
47                $display("-----\n Test Finished \n-----");
48                $display("Total Tests:
49                | Correct Tests$stop;
50                | Error Tests$end
51                ", correct_count + error_count, correct_count, error_count);
52            end
53        endmodule
```

Test Bench:

```
1 import shared_pkg::*;
2 import FIFO_transaction_pkg::*;
3
4 module fifo_tb(fifo_if.test f_if);
5
6     FIFO_transaction transaction_obj;
7
8     initial begin
9
10         transaction_obj = new();
11         //Reset the Design
12         f_if.rst_n = 0;
13         f_if.wr_en = 0;
14         f_if.rd_en = 0;
15         f_if.data_in = '0;
16         repeat (2) @(negedge f_if.clk);
17         f_if.rst_n = 1;
18
19         //A special test for underflow
20         f_if.rst_n = 0;
21         repeat (2) @(negedge f_if.clk);
22         f_if.rst_n = 1;
23         @(posedge f_if.clk);
24
25         repeat (4) begin
26             f_if.rd_en = 1;
27             f_if.wr_en = 0;
28             f_if.data_in = '0;
29
30             -> driving_finished;
31
32             @(posedge f_if.clk);
33         end
34
35         f_if.rd_en = 0;
36         f_if.wr_en = 0;
37
38         // Main stimulus loop
39         repeat (100000) begin
40             assert(transaction_obj.randomize());
41
42             // Drive inputs
43             f_if.wr_en = tr.wr_en;
44             f_if.rd_en = tr.rd_en;
45             f_if.data_in = tr.data_in;
46
47             // trigger after driving inputs
48             -> driving_finished;
49
50             // wait for clock edge before next drive
51             @(negedge f_if.clk);
52         end
53
54         // Signal the monitor to stop
55         test_finished = 1;
56         -> driving_finished;
57
58         // optional delay to allow monitor to print summary
59         repeat(2) @(negedge f_if.clk);
60         $display("TB: Simulation finished driving transactions.");
61
62     end
63
64 endmodule
65
```

Note: Due to Timing mismatch between monitor and test bench I couldn't solve, It will give some incorrect tests even though this is the intended behavior of FIFO

Top Module

```
1  import shared_pkg::*;
2
3  module top();
4
5      bit clk;
6      //Clock generation
7      initial begin
8          clk = 0;
9          forever
10             #1 clk = ~clk;
11      end
12      //Instantiate Interface Module
13      fifo_if f_if(clk);
14
15      //Instantiate Design, Testbench, and monitor
16      FIFO DUT (f_if);
17      fifo_tb tb (f_if);
18      fifo_monitor monitor(f_if);
19
20      //asynchronous reset Assertion
21      property p_reset;
22          @(posedge f_if.clk) (~f_if.rst_n
23          |-> (f_if.overflow == 0 && f_if.underflow == 0));
24      endproperty
25
26      async_reset_assertion: assert property(p_reset);
27      async_reset_coverage : cover property(p_reset);
28
29
30  endmodule
```

Do File

```
1  vlib work
2
3  vlog -f src_files.list +cover -covercells +define+SIM
4
5  vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all -cover
6
7  do wave.do
8
9  run 0
10
11  add wave -position insertpoint \
12  sim:/top/monitor/transaction_obj
13
14  coverage save tb_FIFO.ucdb -onexit -du work.FIFO
15
16  run -all
```

Questasim Waveform:

